

UNSUPERVISED MACHINE LEARNING

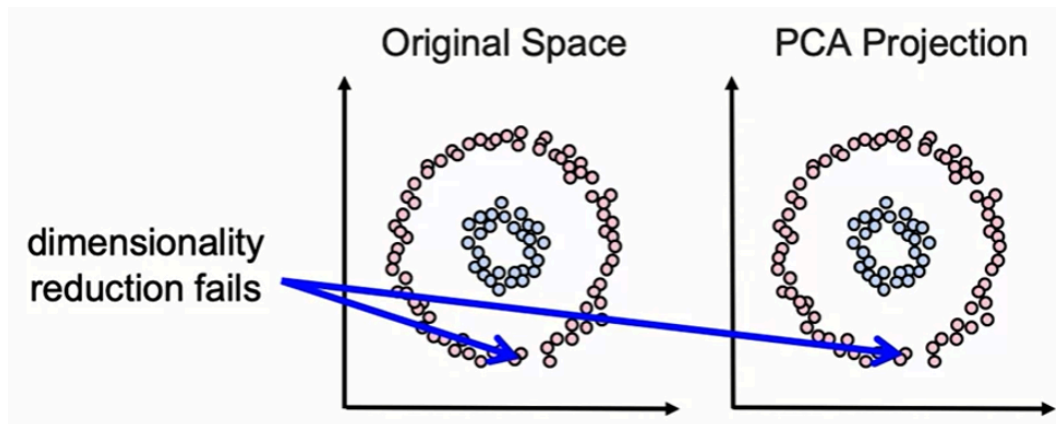
MODULE 5:

NONLINEAR AND DISTANCE-BASED DIMENSIONALITY REDUCTION

TABLE OF CONTENTS

Introduction: Beyond Linear PCA.....	2
Kernel Principal Component Analysis (Kernel PCA).....	2
Multi-Dimensional Scaling (MDS).....	4
Comparison: PCA vs. MDS.....	6
Key Takeaways.....	6

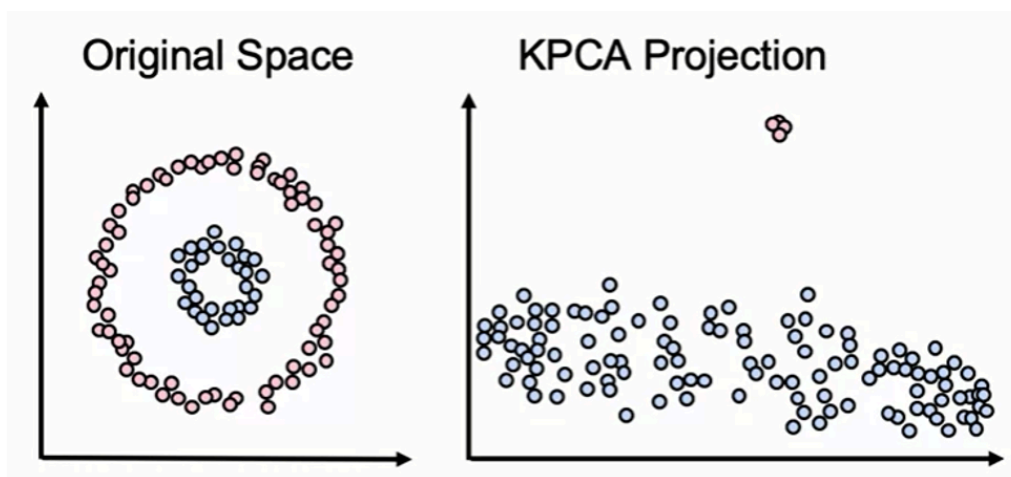
Introduction: Beyond Linear PCA



Concept:

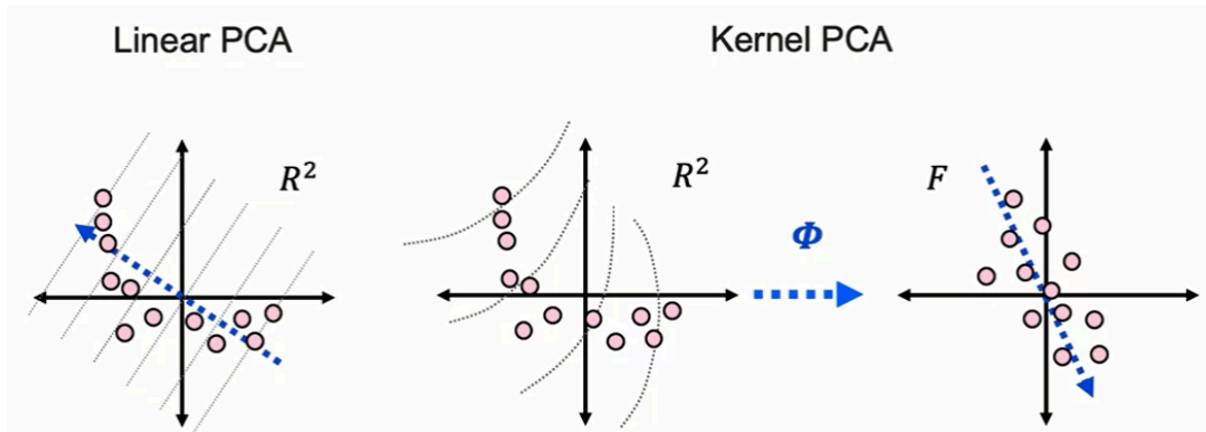
- Traditional PCA uses *linear transformations* to reduce dimensionality, which works well for linearly distributed data.
- However, in many real-world cases, **data relationships are nonlinear**, and linear PCA may fail to capture important structures.
- To solve this, we use **Kernel PCA**, which applies *nonlinear transformations* before performing PCA.

Kernel Principal Component Analysis (Kernel PCA)



Concept

- **Kernel PCA** extends traditional PCA using **kernel methods** (same concept used in SVMs).
- It **maps data to a higher-dimensional feature space** where the data becomes *linearly separable*, then applies standard PCA in that new space.
- This allows PCA to work effectively on **nonlinear data structures**.



Available Kernels

- linear
- poly (polynomial)
- rbf (radial basis function)
- sigmoid
- cosine

Each kernel introduces different transformations depending on data shape and complexity.

Parameters

- **gamma:** Controls the *curvature* or *nonlinearity* (especially in RBF kernels).
- **alpha:** Regularization parameter to avoid overfitting.

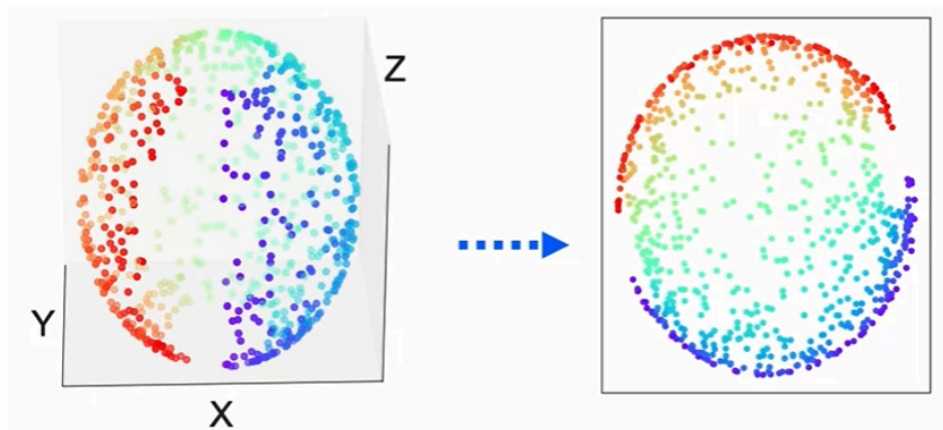
Note: Unlike linear PCA, Kernel PCA may **not perfectly reconstruct** the original data after dimensionality reduction.

```
from sklearn.decomposition import KernelPCA

# Create a Kernel PCA model
kernel_pca = KernelPCA(
    kernel="rbf",
    gamma=10,
    fit_inverse_transform=True,
    alpha=0.1)

# Fit and transform data
kernel_pca.fit(X_train)
score_kernel_pca = kernel_pca.transform(X_test)
```

Multi-Dimensional Scaling (MDS)



Concept

- MDS is another family of *nonlinear dimensionality reduction* algorithms.
- Unlike PCA, which preserves **variance**, MDS focuses on preserving the **distances** (or dissimilarities) between points in the reduced space.
- It can also be used to **visualize relationships** among complex data in 2D or 3D.

Two Main Types of MDS

Metric MDS

- Seeks an embedding that **minimizes the difference** between distances in the original space and reduced space.
- Works directly with distance metrics (e.g., Euclidean, cosine).

$$\min_Y \sum_{i < j} (d_{ij} - \delta_{ij})^2$$

where:

- d_{ij} : distance in reduced space
- δ_{ij} : distance in original space

Non-Metric MDS

- Uses a **monotonic function $f(\cdot)$** to transform distances before minimizing

$$\min_Y \sum_{i < j} (f(d_{ij}) - \delta_{ij})^2$$

- Focuses on preserving the *rank order* of distances rather than their exact values.

```
from sklearn.manifold import MDS

embedding = MDS(
    dissimilarity='precomputed',
    random_state=0,
    n_components=2,
    metric=False # False => Non-Metric MDS)

X_transformed = embedding.fit_transform(X)
```

Comparison: PCA vs. MDS

Aspect	Principal Component Analysis (PCA)	Multidimensional Scaling (MDS)
Objective	Preserve maximal variance	Preserve the pairwise distances
Transformation	Linear transformation	Nonlinear transformation
Output	Orthogonal components	Embedding maintaining geometry
Use Case	Compression, feature extraction	Visualization, Similarity mapping

Other Related Methods (Manifold Learning)

- **Isomap:** Uses nearest neighbors to preserve geodesic distances.
- **t-SNE:** Keeps similar points close and dissimilar points far apart — excellent for visualization.

Key Takeaways

- When data is **nonlinear**, prefer **Kernel PCA** or **MDS** over standard PCA.
- Always **tune kernel type** and **gamma** for Kernel PCA.
- Use **MDS** or **t-SNE** for visualization and pattern discovery.
- Both techniques aim to represent high-dimensional data in a simpler, interpretable way.